

Navigation시스템 도착 예측 모델링

- 도메인의 이해
- 분석대상 데이터 이해
- 데이터탐색(시각화)
- 데이터 전처리
- 파이썬 실습
- 인공지능 활용을 통한 데이터분석 및 전처리
- 머신러닝 모델링
- 딥러닝 모델링
- 파이썬 실습

모델링 프로세스



문제이해
및 분석

데이터수집
데이터병합
시각화
상관분석

결측치 처리
이상치 처리
변수 변환
스케일링

회귀분석
의사결정트리
랜덤포레스트
XGBoost
딥러닝

모델 평가
및 파라미터 조정

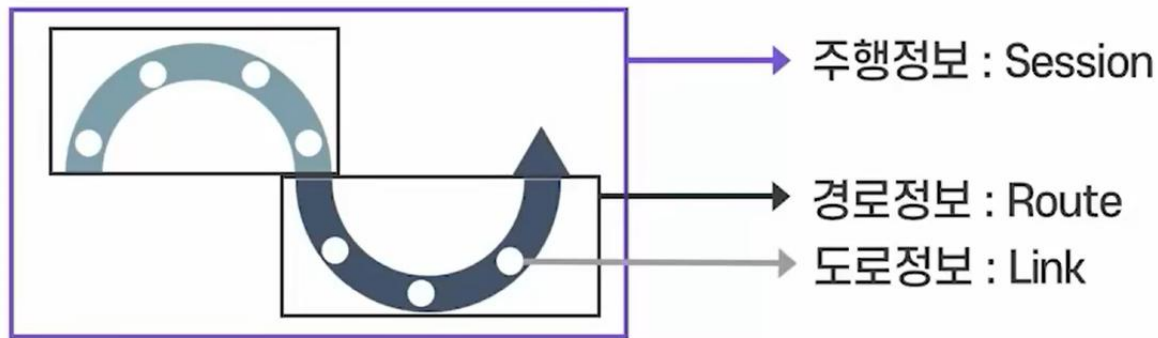
도메인 이해(1/3)

◆ 내비게이션 도착 예측 시간 정확도 향상

- 과거 주행 데이터를 기반으로 navigation 도착시간 예측 정확도 개선

[길 안내 구성요소]

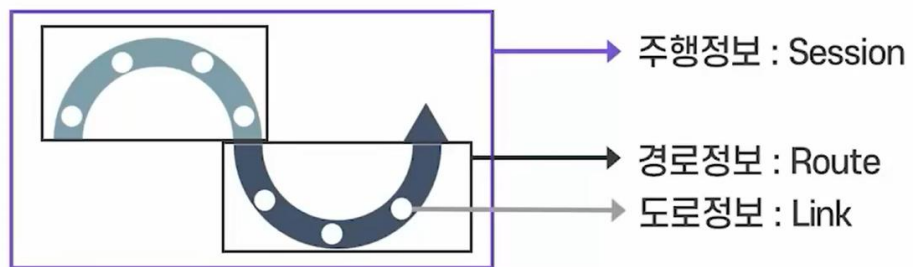
구분	KT 원내비 용어	설명
주행정보	Session	운전자 기준에서 주행의 시작과 끝
경로정보	Route	경로 탐색이 이루어질 때마다 생성(재탐색 시마다 새로 생성)
도로정보	Link	주행경로의 최소단위(도로를 세분화)



도메인 이해(2/3)

[길 안내 구성요소]

구분	KT 원내비 용어	설명
주행정보	Session	운전자 기준에서 주행의 시작과 끝
경로정보	Route	경로 탐색이 이루어질 때마다 생성(재탐색 시마다 새로 생성)
도로정보	Link	주행경로의 최소단위(도로를 세분화)



경로정보

Index	RID	TIME_DEPARTUREDATE	TIME_ARRIVEDATE	_DISTANC	ET	ETA	ETAA
0	router-84875df7fc-b5nxc-7-20255166-0	2020-07-20 05:35:21.000	2020-07-20 05:55:22.599	12914	1201.53	987.625	82.197
1	router-84875df7fc-b5nxc-6-10888379-0	2020-07-20 05:55:22.000	2020-07-20 06:09:06.941	7483	823.817	855.934	96.1015
2	router-84875df7fc-cmkz9-7-45806143-0	2020-07-20 00:13:46.000	2020-07-20 00:22:32.909	8087	526.711	575.955	90.6507
3	router-84875df7fc-scxcj-7-47843617-0	2020-07-20 00:13:43.000	2020-07-20 00:28:42.729	10528	898.582	537.117	59.7738
4	router-84875df7fc-scxcj-3-20061505-0	2020-07-20 00:01:57.000	2020-07-20 00:17:58.169	10636	957.759	786.353	82.1034

추가정보1(경로 출발도시)

Index	RID	level1_pnu	level2_pnu
0	router-84875df7fc-b5nxc-2-2300503-0	경기도	고양시 덕양구
1	router-84875df7fc-scxcj-6-3976613-0	서울특별시	금천구
2	router-84875df7fc-52vz8-4-7281375-0	경기도	화성시
3	router-84875df7fc-scxcj-3-7147622-0	경기도	하남시
4	router-84875df7fc-cmkz9-4-7525466-0	서울특별시	서초구

추가정보2(경로의 신호등 갯수)

Index	RID	signaltype
0	router-84875df7fc-52vz8-0-10002399-0	22
1	router-84875df7fc-52vz8-0-10003032-0	2
2	router-84875df7fc-52vz8-0-10003142-0	20
3	router-84875df7fc-52vz8-0-10012957-0	7
4	router-84875df7fc-52vz8-0-10018039-0	24
5	router-84875df7fc-52vz8-0-10019426-0	16

도메인 이해(3/3)

목표변수 : ET(실주행시간, Estimated Time)

설명변수 : 거리(Distance) + ETA(과거 예측주행시간) + 신호등(Signal) + 요일 + 시간대 + 경유도시

목표변수

Index	RID	TIME_DEPARTUREDATE	TIME_ARRIVEDATE	_DISTANC	ET	ETA	ETAA	level1_pnu	level2_pnu	signaltype
0	router-84875df7fc-b5nxc-7-20255166-0	2020-07-20 05:35:21.000	2020-07-20 05:55:22.599	12914	1201.53	987.625	82.197	경기도	광명시	7
1	router-84875df7fc-b5nxc-6-10888379-0	2020-07-20 05:55:22.000	2020-07-20 06:09:06.941	7483	823.817	855.934	96.1015	서울특별시	영등포구	31
2	router-84875df7fc-cmkz9-7-45806143-0	2020-07-20 00:13:46.000	2020-07-20 00:22:32.909	8087	526.711	575.955	90.6507	경기도	김포시	6
3	router-84875df7fc-scxcj-7-47843617-0	2020-07-20 00:13:43.000	2020-07-20 00:28:42.729	10528	898.582	537.117	59.7738	경기도	광주시	2
4	router-84875df7fc-scxcj-3-20061505-0	2020-07-20 00:01:57.000	2020-07-20 00:17:58.169	10636	957.759	786.353	82.1034	경기도	안산시 상록구	28

*ET:실주행시간
**ETA:예측시간

$$ETAA = (1 - \frac{|ET - ETA|}{ET}) \times 100$$

ETAA(Estimated Time of Arrival Accuracy, 도착예측시간정확도)

실주행시간과 예측주행시간의 차이를 이용한 지표 : 값의 범위 : (0 ~ 100%) => 100에 가까울수록 정확한 예측

전처리 결과

Index	RID	TIME_DEPARTUREDATE	TIME_ARRIVEDATE	_DISTANC	ET	ETA	ETAA	level1_pnu	level2_pnu	signaltype
0	router-84875df7fc-b5nxc-7-20255166-0	2020-07-20 05:35:21.000	2020-07-20 05:55:22.599	12914	1201.53	987.625	82.197	경기도	광명시	7
1	router-84875df7fc-b5nxc-6-10888379-0	2020-07-20 05:55:22.000	2020-07-20 06:09:06.941	7483	823.817	855.934	96.1015	서울특별시	영등포구	31
2	router-84875df7fc-cmkz9-7-45806143-0	2020-07-20 00:13:46.000	2020-07-20 00:22:32.909	8087	526.711	575.955	90.6507	경기도	김포시	6
3	router-84875df7fc-scxcj-7-47843617-0	2020-07-20 00:13:43.000	2020-07-20 00:28:42.729	10528	898.582	537.117	59.7738	경기도	광주시	2
4	router-84875df7fc-scxcj-3-20061505-0	2020-07-20 00:01:57.000	2020-07-20 00:17:58.169	10636	957.759	786.353	82.1034	경기도	안산시 상록구	28

목표변수

df_target DataFrame (55421, 1)

navi_train_feature.csv

df_feature DataFrame (55421, 108)

Index	A_DISTANCE	ETA	signaltype	WEEKDAY_0	WEEKDAY_1	WEEKDAY_2	WEEKDAY_3	WEEKDAY_4	HOUR_0	HOUR_1	HOUR_2
0	0.826167	0.216722	0.0648148	1	0	0	0	0	0	0	0
1	0.373583	0.185241	0.287037	1	0	0	0	0	0	0	0
2	0.423017	0.118211	0.0555556	1	0	0	0	0	1	0	0

navi_train_target.csv

Index	ET
0	1201.53
1	823.817
2	526.711

navi_eval_feature.csv

eval_feature DataFrame (53732, 108)

Index	A_DISTANCE	ETA	signaltype	WEEKDAY_0	WEEKDAY_1	WEEKDAY_2	WEEKDAY_3	WEEKDAY_4	HOUR_0
0	0.148333	0.101265	0.166667	1	0	0	0	0	1
1	0.375417	0.0775983	0.00925926	1	0	0	0	0	1
2	0.94575	0.195422	0.166667	1	0	0	0	0	1

navi_eval_target.csv

Index	ET
0	555.547
1	397.209
2	720.646

eval_target DataFrame (53732, 1)

데이터분할

df_feature - DataFrame

Index	A_DISTANCE	ETA	signaltype	VEHICLE
0	0.826167	0.216722	0.0648148	1
1	0.373583	0.185241	0.287037	1
2	0.423917	0.118311	0.0555556	1
3	0.627333	0.109026	0.0185185	1
4	0.636333	0.168607	0.259259	1

train_x - DataFrame

Index	A_DISTANCE	ETA	signaltype	VEHICLE
50823	0.37575	0.20914	0.0648148	0
13793	0.103833	0.164964	0.157407	0
53541	0.0449167	0.103814	0.0185185	0

훈련용

test_x - DataFrame

Index	A_DISTANCE	ETA	signaltype	VEHICLE
51147	0.615917	0.281116	0.166667	0
49616	0.0265	0.139759	0.037037	0
9314	0.207	0.177286	0.240741	0

테스트 용

df_target - DataFrame

Index	ET
0	1201.53
1	823.817
2	526.711
3	898.582
4	957.759

train_y - DataFrame

Index	ET
50823	1160
13793	623.274
53541	600.057

train_y DataFrame (44336, 1)

훈련용

test_y - DataFrame

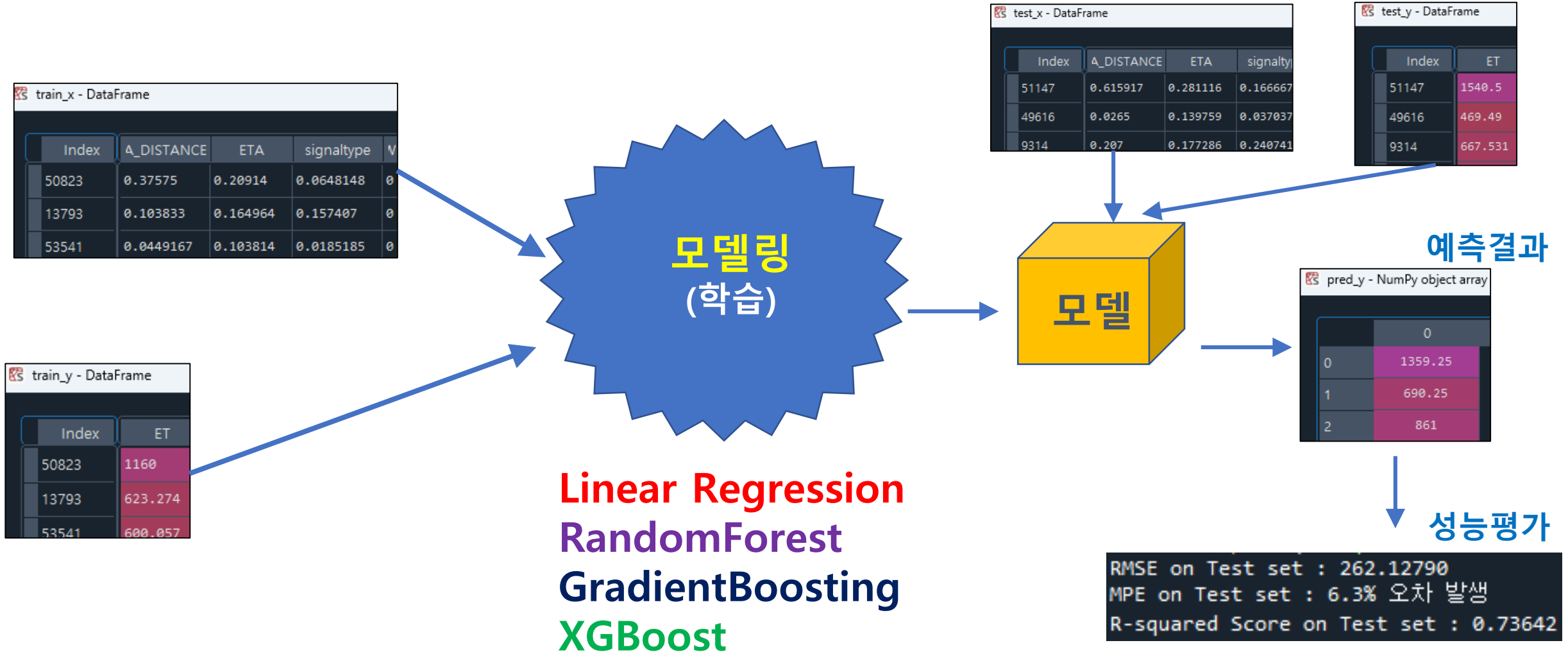
Index	ET
51147	1540.5
49616	469.49
9314	667.531

test_y DataFrame (11085, 1)

테스트 용

df_target DataFrame (55421, 1)

머신러닝 기반 모델링(1/2)



머신러닝 주요 알고리즘 분류



RMSE

◆ RMSE(Root Mean Squared Error, 평균제곱근오차)

- 예측값이 숫자형인 경우의 평가지표
- 모델의 예측값이 실제값과 얼마나 차이 나는지(오차) 를 나타내는 지표
- “예측이 실제와 얼마나 정확하게 일치하나?”를 평균적인 거리로 측정

RMSE on Test set : 262.12790
MPE on Test set : 6.3% 오차 발생
R-squared Score on Test set : 0.73642

공식

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

기호	의미
y_i	실제값 (Observed Value)
$\hat{y}_i$	예측값 (Predicted Value)
n	데이터 개수

MAE

예시

학생	실제 점수 (Y)	예측 점수 ($\hat{Y}$)	오차(e = Y - $\hat{Y}$)	오차제곱
A	80	78	+2	4
B	70	72	-2	4
C	90	85	+5	25
D	60	65	-5	25

$$RMSE = \sqrt{(4 + 4 + 25 + 25)/4} = \sqrt{14.5} \approx 3.81$$

⇒ RMSE = 3.81점

“오차의 제곱의 평균”을 구한 뒤 제곱근을 취한 값

예측값은 실제 점수에서 평균적으로 약 ±3.8점 오차가 발생한다는 뜻

MPE

MPE 는 Mean Percentage Error (평균 백분율 오차) 의 약자

모델의 예측값이 실제값에 비해 평균적으로 몇 퍼센트나 차이 나는지를 보여주는 지표

$$MPE = \frac{100}{n} \sum_{i=1}^n \frac{y_i - \hat{y}_i}{y_i}$$

RMSE on Test set : 262.12790
MPE on Test set : 6.3% 오차 발생
R-squared Score on Test set : 0.73642

실제값(y)	예측값($\hat{y}$)	오차($y - \hat{y}$)	비율($(y - \hat{y})/y$)	(%)
100	90	+10	+0.10	+10%
200	220	-20	-0.10	-10%
300	270	+30	+0.10	+10%

$$MPE = \frac{(+10\% - \downarrow -10\% + 10\%)}{3} = +3.3\%$$

R제곱(R-Squared)

회귀모형 진단

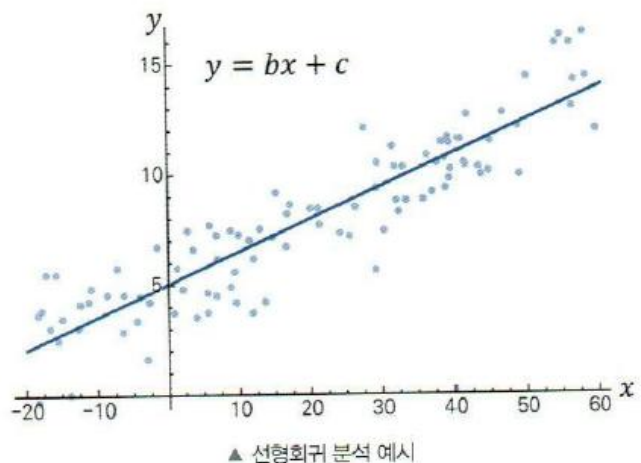
1 적합도 검정(Goodness-of-fit Test)

- 회귀모형이 데이터를 얼마나 잘 설명하는가 → 결정계수 R^2

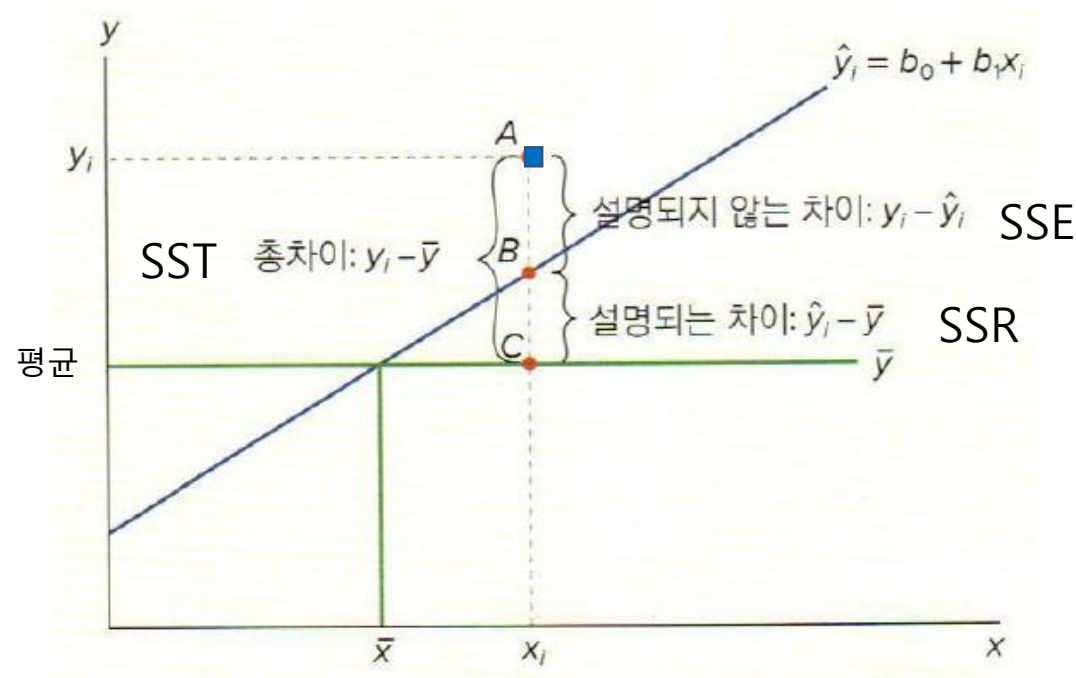
$$R^2 = \frac{SSR}{SST}$$

- SSR (Regression Sum of Squares): 회귀로 설명되는 변동
- SSE (Error Sum of Squares): 회귀로 설명되지 않는 변동
- SST (Total Sum of Squares): 전체 변동

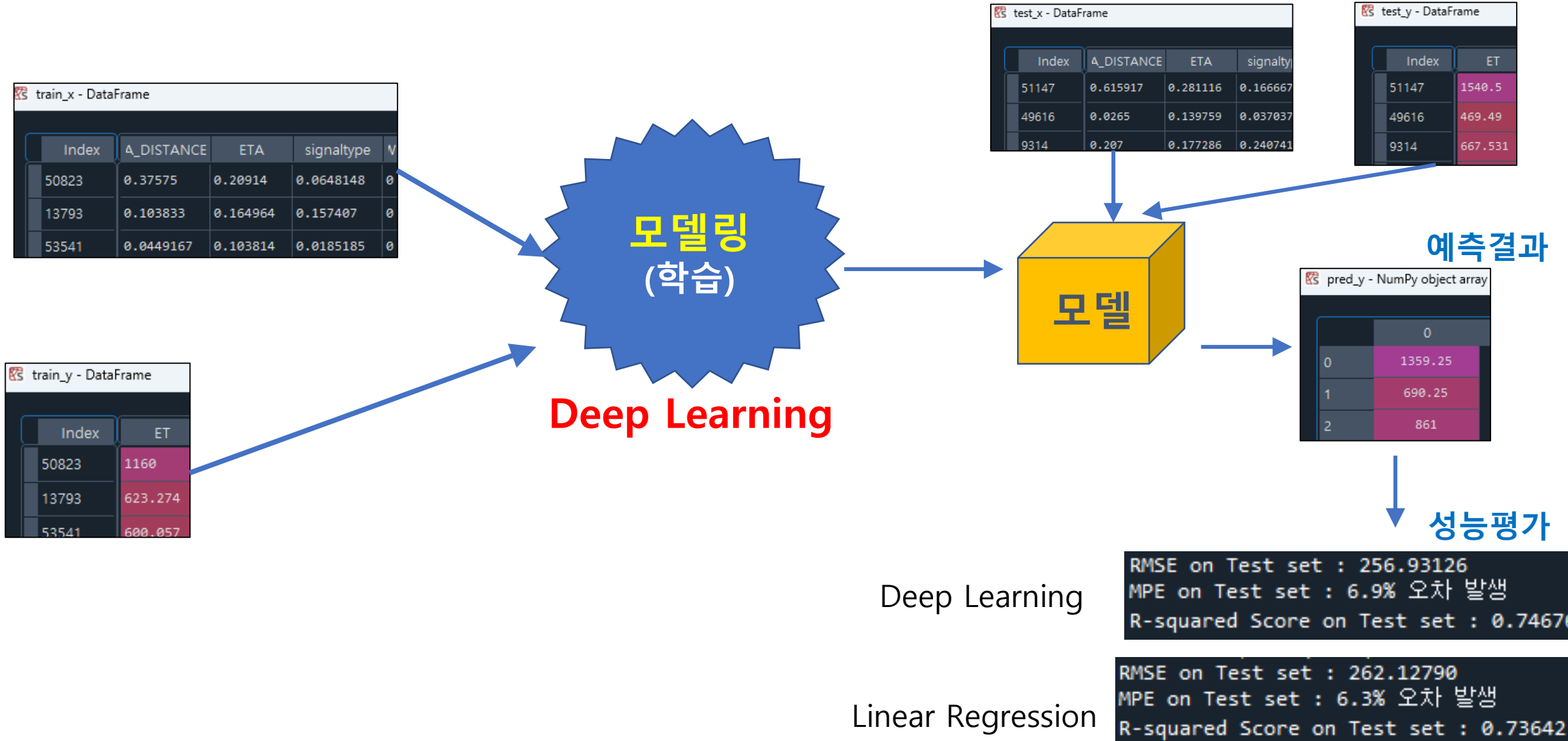
범위: 0~1 (1에 가까울수록 좋은 모델)



RMSE on Test set : 262.12790
MPE on Test set : 6.3% 오차 발생
R-squared Score on Test set : 0.73642



딥러닝 기반 모델링(2/4)



딥러닝 기반 모델링(3/4)

다음 조건에 따라 딥러닝 모델을 만들고 모델의 성능평가를 해봐
<조건>

1. 학습데이터

feature 데이터: navi_train_feature.csv(첨부)

target 데이터: navi_train_target.csv(첨부)

2. 모델구조

- 은닉층 2개(유닛은 64개)

3. 성능평가

- RMSE, MPE, R제곱



현재 내 실행 환경에는 TensorFlow 가 설치되어 있지 않아서 🤖 직접 모델을 학습시킬 수는 없어.
하지만 Mario가 바로 Colab이나 VSCode에서 돌릴 수 있도록 완전한 실행 코드 예시를 줄게 🙌

✅ 딥러닝 모델 학습 + 성능평가 코드 (RMSE, MPE, R²)

python

📄 코드 복사

```
# -----  
# 📄 라이브러리 불러오기  
# -----  
import pandas as pd  
import numpy as np  
from tensorflow import keras  
from tensorflow.keras import layers  
from sklearn.metrics import mean_squared_error, r2_score  
from math import sqrt
```



딥러닝 기반 모델링(4/4)

다음 조건에 따라 딥러닝 모델을 만들고 모델의 성능평가를 해봐
<조건>

1. 학습데이터

feature 데이터: navi_train_feature.csv(첨부)

target 데이터: navi_train_target.csv(첨부)

2. 모델구조

- 은닉층 2개(유닛은 64개)

3. 성능평가

- RMSE, MPE, R제곱



```
import pandas as pd
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import mean_squared_error, r2_score
from math import sqrt

# -----
# ❷ 데이터 불러오기
# -----
X = pd.read_csv("navi_train_feature.csv")
y = pd.read_csv("navi_train_target.csv")

# -----
# ❸ 모델 구성
# -----
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=[X.shape[1]]),
    layers.Dense(64, activation='relu'),
    layers.Dense(1)
])

# -----
# ❹ 모델 컴파일 및 학습
# -----
model.compile(optimizer='adam', loss='mse', metrics=['mae', 'mse'])
history = model.fit(X, y, epochs=30, validation_split=0.2, verbose=1)

# -----
# ❺ 예측 및 성능평가
# -----
pred_y = model.predict(X)

# RMSE
rmse = sqrt(mean_squared_error(y, pred_y))

# MPE (평균 백분율 오차)
mpe = np.mean((y.values.flatten() - pred_y.flatten()) / y.values.flatten()) * 100

# R² (결정계수)
r2 = r2_score(y, pred_y)

print(f"❶ RMSE: {rmse:.4f}")
print(f"❶ MPE : {mpe:.2f}%")
print(f"❶ R² : {r2:.4f}")

# -----
# ❻ 학습 손실 시각화
# -----
import matplotlib.pyplot as plt

plt.style.use("ggplot")
plt.figure()
plt.plot(history.history["loss"], label="train_loss")
plt.plot(history.history["val_loss"], label="val_loss")
plt.title("Training Loss (MSE)")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```


파이썬 실습

파이썬 가상환경 생성 및 텐서플로 등 설치

```
# 아나콘다 Power Shell을 관리자모드로 실행 후,다음 명령어를 순차적으로 입력
conda create -n tfenv python=3.10
conda activate tfenv
pip install tensorflow
pip install numpy==1.26.4
pip install pandas
pip install matplotlib
pip install seaborn
pip install scikit-learn
pip install xgboost
pip install spyder
spyder 실행
```

감사합니다.